Contents

Python Basics Cheat Sheet **1**
Tabla de Contenidos . . . 1
1. Variables y Asignación . . . 1
2. Tipos de Datos Primitivos . . . 2
3. Operadores . . . 4
4. Strings . . . 5
5. Input/Output . . . 8
6. Condicionales . . . 9
7. Bucles . . . 11
8. Funciones . . . 13
9. Type Hints . . . 15
10. Errores Comunes . . . 17
Recursos Relacionados . . . 19

Python Basics Cheat Sheet

Python 3.13+ | Referencia rápida de fundamentos

Tabla de Contenidos

1. Variables y Asignación
2. Tipos de Datos Primitivos
3. Operadores
4. Strings
5. Input/Output
6. Condicionales
7. Bucles
8. Funciones
9. Type Hints
10. Errores Comunes

---

1. Variables y Asignación

Asignación Básica

```python
# Asignación simple
name = "Python"
age = 30
price = 19.99
is_active = True

# Asignación múltiple
x, y, z = 1, 2, 3
a = b = c = 0

# Intercambio de valores (swap)
x, y = y, x

# Desempaquetado con *
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2,3,4], last=5
```

Walrus Operator := (Python 3.8+)

```python
# Asignar y usar en la misma expresión
if (n := len(data)) > 10:
    print(f"Lista grande: {n} elementos")

# En comprehensions
results = [y for x in data if (y := process(x)) is not None]

# En bucles while
while (line := file.readline()):
    process(line)
```

Convenciones de Nomenclatura

Tipo	Convención	Ejemplo
Variables	`snake_case`	`user_name`, `total_count`
Constantes	`UPPER_SNAKE_CASE`	`MAX_SIZE`, `API_KEY`
Clases	`PascalCase`	`UserAccount`, `HttpClient`
Funciones	`snake_case`	`get_user()`, `calculate_total()`
Privados	`_prefijo`	`_internal_value`
Name mangling	`__prefijo`	`__private_attr`

---

2. Tipos de Datos Primitivos

Resumen de Tipos

Tipo	Ejemplo	Mutable	Descripción
`int`	`42`, `-7`, `0b1010`		Enteros sin límite
`float`	`3.14`, `-0.001`, `1e-5`		Punto flotante (64-bit)
`bool`	`True`, `False`		Booleano
`str`	`"hello"`, `'world'`		Cadena de texto
`None`	`None`		Valor nulo
`complex`	`3+4j`		Número complejo

Enteros (`int`)

```python
# Diferentes bases
decimal = 255
binario = 0b11111111    # 255
octal = 0o377           # 255
hexadecimal = 0xFF      # 255

# Separadores para legibilidad (Python 3.6+)
billion = 1_000_000_000
hex_color = 0xFF_FF_FF

# Operaciones
abs(-5)          # 5
pow(2, 10)       # 1024
divmod(17, 5)    # (3, 2) -> (cociente, resto)
```

Flotantes (`float`)

```
# Notación científica
avogadro = 6.022e23
planck = 6.626e-34

# Funciones útiles
round(3.14159, 2)    # 3.14
import math
math.floor(3.7)      # 3
math.ceil(3.2)       # 4
math.trunc(-3.7)     # -3

#  Precisión flotante
0.1 + 0.2 == 0.3     # False!
math.isclose(0.1 + 0.2, 0.3)  # True
```

Booleanos (`bool`)

```
# Valores truthy y falsy
# Falsy: False, None, 0, 0.0, "", [], {}, set()
# Truthy: todo lo demás

bool([])        # False
bool([1, 2])    # True
bool("")        # False
bool("hello")   # True
bool(0)         # False
bool(42)        # True

# Operadores booleanos
True and False   # False
True or False    # True
not True         # False

# Cortocircuito (short-circuit)
x = None
result = x or "default"  # "default"
```

Conversiones de Tipo

```
# A entero
int("42")        # 42
int(3.9)         # 3 (trunca)
int("1010", 2)   # 10 (base 2)
int("ff", 16)    # 255 (base 16)

# A flotante
float("3.14")    # 3.14
float(42)        # 42.0

# A string
str(42)          # "42"
str(3.14)        # "3.14"
str(True)        # "True"
```

```
# A booleano
bool(1)          # True
bool(0)          # False
bool("text")     # True
bool("")         # False
```

---

3. Operadores

Operadores Aritméticos

Operador	Descripción	Ejemplo	Resultado
`+`	Suma	`5 + 3`	`8`
`-`	Resta	`5 - 3`	`2`
`*`	Multiplicación	`5 * 3`	`15`
`/`	División	`7 / 2`	`3.5`
`//`	División entera	`7 // 2`	`3`
`%`	Módulo (resto)	`7 % 2`	`1`
`**`	Potencia	`2 ** 3`	`8`

Operadores de Comparación

Operador	Descripción	Ejemplo
`==`	Igual	`5 == 5` → `True`
`!=`	Diferente	`5 != 3` → `True`
`<`	Menor que	`3 < 5` → `True`
`>`	Mayor que	`5 > 3` → `True`
`<=`	Menor o igual	`3 <= 3` → `True`
`>=`	Mayor o igual	`5 >= 5` → `True`

```
# Comparaciones encadenadas
1 < x < 10         # Equivale a: 1 < x and x < 10
a == b == c        # Equivale a: a == b and b == c
```

Operadores Lógicos

Operador	Descripción	Ejemplo
`and`	Y lógico	`True and False` → `False`
`or`	O lógico	`True or False` → `True`
`not`	Negación	`not True` → `False`

```
# Retornan valores, no solo True/False
"a" and "b"       # "b" (último evaluado)
"" and "b"        # "" (primero falsy)
"a" or "b"        # "a" (primero truthy)
"" or "b"         # "b" (primero truthy)
```

Operadores de Identidad y Membresía

```python
# Identidad (mismo objeto en memoria)
a is b            # True si son el mismo objeto
a is not b        # True si son objetos diferentes

#  Usar == para comparar valores, is para None
x == None         #  Funciona pero no recomendado
x is None         #  Correcto

# Membresía
"a" in "abc"            # True
5 in [1, 2, 3, 4, 5]    # True
"key" in {"key": 1}     # True (busca en keys)
"x" not in "abc"        # True
```

Operadores de Asignación Compuesta

Operador	Equivalente
`x += 5`	`x = x + 5`
`x -= 5`	`x = x - 5`
`x *= 5`	`x = x * 5`
`x /= 5`	`x = x / 5`
`x //= 5`	`x = x // 5`
`x %= 5`	`x = x % 5`
`x **= 5`	`x = x ** 5`

Operadores Bitwise

Operador	Descripción	Ejemplo
`&`	AND bit a bit	`5 & 3` → `1`
`|`	OR bit a bit	`5 | 3` → `7`
`^`	XOR bit a bit	`5 ^ 3` → `6`
`~`	NOT bit a bit	`~5` → `-6`
`<<`	Shift izquierda	`5 << 1` → `10`
`>>`	Shift derecha	`5 >> 1` → `2`

---

4. Strings

Creación de Strings

```python
# Comillas simples o dobles
s1 = 'Hello'
s2 = "World"

# Multilinea
s3 = """Línea 1
Línea 2
Línea 3"""

# Raw strings (sin escapar)
path = r"C:\Users\name\file.txt"
```

```
# Bytes
b = b"hello"  # bytes literal
```

f-Strings (Formateo Moderno)

```
name = "Alice"
age = 30

# Básico
f"Hola, {name}!"                    # "Hola, Alice!"

# Expresiones
f"En 5 años tendrás {age + 5}"      # "En 5 años tendrás 35"

# Formateo numérico
pi = 3.14159
f"{pi:.2f}"                         # "3.14"
f"{1000000:,}"                      # "1,000,000"
f"{1000000:_}"                      # "1_000_000"
f"{255:08b}"                        # "11111111" (binario 8 dígitos)
f"{255:02x}"                        # "ff" (hex 2 dígitos)

# Alineación
f"{'texto':>10}"                    # "     texto" (derecha)
f"{'texto':<10}"                    # "texto     " (izquierda)
f"{'texto':^10}"                    # "  texto   " (centro)
f"{'texto':*^10}"                   # "**texto***" (relleno)

# Debug con = (Python 3.8+)
x = 42
f"{x=}"                             # "x=42"
f"{x = }"                           # "x = 42"
f"{x*2=}"                           # "x*2=84"

# Porcentajes
ratio = 0.756
f"{ratio:.1%}"                      # "75.6%"

# Fechas
from datetime import datetime
now = datetime.now()
f"{now:%Y-%m-%d %H:%M}"             # "2026-01-02 14:30"
```

Métodos de String Más Usados

```
s = "Hello, World!"

# Búsqueda
s.find("World")       # 7 (índice, -1 si no existe)
s.index("World")      # 7 (índice, error si no existe)
s.count("l")          # 3
s.startswith("Hello") # True
s.endswith("!")       # True
"World" in s          # True
```

```
# Verificación de contenido
"abc123".isalnum()    # True (alfanumérico)
"abc".isalpha()       # True (solo letras)
"123".isdigit()       # True (solo dígitos)
"  ".isspace()        # True (solo espacios)
"hello".islower()     # True
"HELLO".isupper()     # True
"Hello World".istitle()  # True
```

Búsqueda y Verificación

```
s = "  Hello, World!  "

# Mayúsculas/Minúsculas
s.lower()             # "  hello, world!  "
s.upper()             # "  HELLO, WORLD!  "
s.capitalize()        # "  hello, world!  "
s.title()             # "  Hello, World!  "
s.swapcase()          # "  hELLO, wORLD!  "

# Espacios
s.strip()             # "Hello, World!"
s.lstrip()            # "Hello, World!  "
s.rstrip()            # "  Hello, World!"

# Reemplazo
s.replace("World", "Python")  # "  Hello, Python!  "

# Alineación
"hi".center(10)       # "    hi    "
"hi".ljust(10)        # "hi        "
"hi".rjust(10)        # "        hi"
"42".zfill(5)         # "00042"
```

Transformación

```
# Split
"a,b,c".split(",")           # ["a", "b", "c"]
"a  b  c".split()            # ["a", "b", "c"] (por espacios)
"a,b,c".split(",", 1)        # ["a", "b,c"] (máx. 1 split)
"line1\nline2".splitlines()  # ["line1", "line2"]

# Join
",".join(["a", "b", "c"])    # "a,b,c"
" ".join(["Hello", "World"]) # "Hello World"
"\n".join(["line1", "line2"]) # "line1\nline2"

# Partition
"a=b=c".partition("=")       # ("a", "=", "b=c")
"a=b=c".rpartition("=")      # ("a=b", "=", "c")
```

División y Unión

Slicing de Strings

```
s = "Python"

# Índices positivos:  0  1  2  3  4  5
# Índices negativos: -6 -5 -4 -3 -2 -1

s[0]        # "P"
s[-1]       # "n"
s[1:4]      # "yth"
s[:3]       # "Pyt"
s[3:]       # "hon"
s[::2]      # "Pto" (cada 2)
s[::-1]     # "nohtyP" (reverso)
s[-3:]      # "hon" (últimos 3)
s[:-3]      # "Pyt" (excepto últimos 3)
```

Caracteres Especiales (Escape)

Secuencia	Descripción
`\n`	Nueva línea
`\t`	Tabulador
`\\`	Barra invertida
`\'`	Comilla simple
`\"`	Comilla doble
`\r`	Retorno de carro
`\0`	Carácter nulo

---

5. Input/Output

Input

```
# Entrada básica (siempre retorna str)
name = input("Ingresa tu nombre: ")

# Convertir a otros tipos
age = int(input("Edad: "))
price = float(input("Precio: "))

# Múltiples valores en una línea
x, y = input("x y: ").split()
x, y = int(x), int(y)

# Usando map
x, y = map(int, input("x y: ").split())

# Lista de números
numbers = list(map(int, input().split()))
```

Print

```
# Básico
print("Hello, World!")
```

```python
# Múltiples argumentos
print("a", "b", "c")          # a b c

# Separador personalizado
print("a", "b", "c", sep="-") # a-b-c
print("a", "b", "c", sep="")  # abc

# Sin salto de línea
print("Loading", end="")
print("...")                   # Loading...

# Imprimir a archivo
with open("output.txt", "w") as f:
    print("Hello", file=f)

# Flush inmediato
print("Processing...", flush=True)
```

Formateo de Output

```python
# f-strings (recomendado)
name, age = "Alice", 30
print(f"{name} tiene {age} años")

# .format()
print("{} tiene {} años".format(name, age))
print("{0} tiene {1} años".format(name, age))
print("{name} tiene {age} años".format(name=name, age=age))

# % (legacy, no recomendado)
print("%s tiene %d años" % (name, age))
```

---

6. Condicionales

if / elif / else

```python
x = 10

# Básico
if x > 0:
    print("Positivo")
elif x < 0:
    print("Negativo")
else:
    print("Cero")

# Una línea (ternario)
result = "par" if x % 2 == 0 else "impar"

# Ternario anidado (evitar si es complejo)
sign = "+" if x > 0 else "-" if x < 0 else "0"
```

Match Statement (Python 3.10+)

```python
def handle_command(command: str) -> str:
    match command:
        case "start":
            return "Iniciando..."
        case "stop":
            return "Deteniendo..."
        case "pause" | "hold":  # OR pattern
            return "Pausando..."
        case _:  # Default (wildcard)
            return "Comando desconocido"

# Con guardas (guards)
def classify_number(n: int) -> str:
    match n:
        case 0:
            return "cero"
        case n if n < 0:
            return "negativo"
        case n if n % 2 == 0:
            return "par positivo"
        case _:
            return "impar positivo"

# Desempaquetado en match
def process_point(point):
    match point:
        case (0, 0):
            return "origen"
        case (x, 0):
            return f"en eje X: {x}"
        case (0, y):
            return f"en eje Y: {y}"
        case (x, y):
            return f"punto ({x}, {y})"

# Match con diccionarios
def process_response(response: dict):
    match response:
        case {"status": 200, "data": data}:
            return f"Éxito: {data}"
        case {"status": 404}:
            return "No encontrado"
        case {"status": status} if status >= 500:
            return f"Error del servidor: {status}"
        case _:
            return "Respuesta desconocida"
```

Truthy y Falsy en Condicionales

```python
# Valores Falsy (evalúan a False)
if not False: pass
if not None: pass
if not 0: pass
if not 0.0: pass
```

```python
if not "": pass
if not []: pass
if not {}: pass
if not set(): pass

# Uso idiomático
items = []
if items:  # En lugar de: if len(items) > 0
    process(items)

name = ""
if name:  # En lugar de: if name != ""
    greet(name)

# Valor por defecto
name = user_input or "Anónimo"
```

---

7. Bucles

for Loop

```python
# Iterar sobre secuencia
for char in "Python":
    print(char)

for item in [1, 2, 3]:
    print(item)

# range()
for i in range(5):          # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 8):       # 2, 3, 4, 5, 6, 7
    print(i)

for i in range(0, 10, 2):   # 0, 2, 4, 6, 8
    print(i)

for i in range(10, 0, -1):  # 10, 9, 8, ..., 1
    print(i)

# enumerate (índice + valor)
fruits = ["apple", "banana", "cherry"]
for i, fruit in enumerate(fruits):
    print(f"{i}: {fruit}")

for i, fruit in enumerate(fruits, start=1):
    print(f"{i}: {fruit}")

# zip (iteración paralela)
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
for name, age in zip(names, ages):
    print(f"{name}: {age}")
```

```python
# zip con desempaquetado extendido
for a, b, *rest in zip(list1, list2, list3, list4):
    print(a, b, rest)
```

while Loop

```python
# Básico
count = 0
while count < 5:
    print(count)
    count += 1

# Con condición de salida
while True:
    user_input = input("Comando: ")
    if user_input == "quit":
        break
    process(user_input)

# Con walrus operator
while (line := input(">>> ")) != "quit":
    print(f"Procesando: {line}")
```

Control de Flujo

```python
# break - salir del bucle
for i in range(10):
    if i == 5:
        break
    print(i)  # 0, 1, 2, 3, 4

# continue - siguiente iteración
for i in range(5):
    if i == 2:
        continue
    print(i)  # 0, 1, 3, 4

# else en bucles (se ejecuta si NO hubo break)
for item in items:
    if item == target:
        print("Encontrado!")
        break
else:
    print("No encontrado")

# pass - placeholder
for i in range(5):
    pass  # TODO: implementar
```

Comprensiones (Comprehensions)

```python
# List comprehension
squares = [x**2 for x in range(10)]
evens = [x for x in range(20) if x % 2 == 0]
```

```python
pairs = [(x, y) for x in range(3) for y in range(3)]

# Dict comprehension
square_dict = {x: x**2 for x in range(5)}
# {0: 0, 1: 1, 2: 4, 3: 9, 4: 16}

filtered = {k: v for k, v in data.items() if v > 0}

# Set comprehension
unique_lengths = {len(word) for word in words}

# Generator expression (lazy, memoria eficiente)
sum_squares = sum(x**2 for x in range(1000000))

# Comprehension con walrus
results = [y for x in data if (y := expensive(x)) is not None]

# Comprehension anidada (aplanar lista)
matrix = [[1, 2], [3, 4], [5, 6]]
flat = [num for row in matrix for num in row]
# [1, 2, 3, 4, 5, 6]
```

---

8. Funciones

Definición Básica

```python
# Sin parámetros
def greet():
    print("Hello!")

# Con parámetros
def greet(name: str) -> None:
    print(f"Hello, {name}!")

# Con valor de retorno
def add(a: int, b: int) -> int:
    return a + b

# Múltiples retornos
def divide(a: int, b: int) -> tuple[int, int]:
    return a // b, a % b

quotient, remainder = divide(17, 5)
```

Parámetros

```python
# Parámetros por defecto
def greet(name: str = "World") -> str:
    return f"Hello, {name}!"

#  NUNCA usar mutables como default
def bad(items: list = []):  #  Bug!
    items.append(1)
    return items
```

```python
def good(items: list | None = None):  #  Correcto
    if items is None:
        items = []
    items.append(1)
    return items

# Argumentos posicionales y keyword
def func(a, b, c):
    pass

func(1, 2, 3)           # Posicional
func(a=1, b=2, c=3)     # Keyword
func(1, c=3, b=2)       # Mixto

# Solo posicionales (/) y solo keyword (*)
def func(pos_only, /, standard, *, kw_only):
    pass

func(1, 2, kw_only=3)      #
func(1, standard=2, kw_only=3)  #
func(pos_only=1, ...)      #  Error
func(1, 2, 3)              #  Error
```

*args y **kwargs

```python
# *args - tupla de argumentos posicionales
def sum_all(*numbers: int) -> int:
    return sum(numbers)

sum_all(1, 2, 3, 4)  # 10

# **kwargs - diccionario de argumentos keyword
def print_info(**kwargs: str) -> None:
    for key, value in kwargs.items():
        print(f"{key}: {value}")

print_info(name="Alice", age="30")

# Combinación
def func(a, b, *args, **kwargs):
    pass

# Desempaquetado al llamar
numbers = [1, 2, 3]
sum_all(*numbers)  # Equivale a sum_all(1, 2, 3)

config = {"name": "Alice", "age": 30}
print_info(**config)  # Equivale a print_info(name="Alice", age=30)
```

Funciones Lambda

```python
# Función anónima de una línea
square = lambda x: x ** 2
add = lambda a, b: a + b
```

```python
# Uso común con funciones de orden superior
numbers = [3, 1, 4, 1, 5, 9]
sorted(numbers, key=lambda x: -x)  # Orden descendente

# Con filter y map
evens = list(filter(lambda x: x % 2 == 0, numbers))
squares = list(map(lambda x: x ** 2, numbers))

# Condicional en lambda
sign = lambda x: "+" if x > 0 else "-" if x < 0 else "0"
```

Docstrings

```python
def calculate_area(radius: float) -> float:
    """
    Calcula el área de un círculo.

    Args:
        radius: El radio del círculo en unidades arbitrarias.

    Returns:
        El área del círculo.

    Raises:
        ValueError: Si el radio es negativo.

    Examples:
        >>> calculate_area(1.0)
        3.14159...
        >>> calculate_area(0)
        0.0
    """
    if radius < 0:
        raise ValueError("El radio no puede ser negativo")
    return 3.14159 * radius ** 2
```

---

9. Type Hints

Tipos Básicos

```python
# Variables
name: str = "Alice"
age: int = 30
price: float = 19.99
is_active: bool = True
nothing: None = None

# Funciones
def greet(name: str) -> str:
    return f"Hello, {name}!"

def process() -> None:  # No retorna nada
    print("Processing...")
```

Tipos Compuestos (Python 3.9+)

```python
# Listas
names: list[str] = ["Alice", "Bob"]
matrix: list[list[int]] = [[1, 2], [3, 4]]

# Diccionarios
ages: dict[str, int] = {"Alice": 30, "Bob": 25}
config: dict[str, str | int] = {"name": "app", "port": 8080}

# Tuplas
point: tuple[int, int] = (10, 20)
record: tuple[str, int, float] = ("Alice", 30, 1.65)
variable_tuple: tuple[int, ...] = (1, 2, 3, 4)  # Longitud variable

# Sets
unique_ids: set[int] = {1, 2, 3}
```

Union y Optional

```python
# Union con | (Python 3.10+)
def process(value: int | str) -> str:
    return str(value)

# Union tradicional
from typing import Union
def process(value: Union[int, str]) -> str:
    return str(value)

# Optional (equivale a X | None)
def find_user(id: int) -> str | None:
    return users.get(id)

# Tradicional
from typing import Optional
def find_user(id: int) -> Optional[str]:
    return users.get(id)
```

Tipos Avanzados

```python
from typing import Callable, TypeVar, Generic

# Callable (funciones como parámetros)
def apply(func: Callable[[int, int], int], a: int, b: int) -> int:
    return func(a, b)

# TypeVar (genéricos)
T = TypeVar('T')

def first(items: list[T]) -> T | None:
    return items[0] if items else None

# Literal (valores específicos)
from typing import Literal

def set_mode(mode: Literal["read", "write", "append"]) -> None:
```

```python
    pass

# TypedDict
from typing import TypedDict

class User(TypedDict):
    name: str
    age: int
    email: str | None
```

Type Aliases

```python
# Alias simple
UserId = int
Username = str

# Alias compuesto
UserDict = dict[str, str | int | None]
Coordinate = tuple[float, float]
Matrix = list[list[int]]

# Usar los alias
def get_user(user_id: UserId) -> UserDict:
    pass

def distance(p1: Coordinate, p2: Coordinate) -> float:
    pass
```

---

10. Errores Comunes

Errores de Sintaxis

```python
# Olvidar dos puntos
if x > 5     #  SyntaxError
if x > 5:    #

# Indentación incorrecta
if True:
print("hi")  #  IndentationError
    print("hi")  #

# Paréntesis sin cerrar
print("hello"  #  SyntaxError
print("hello") #
```

Errores de Tipo

```python
# Concatenar string con número
"Edad: " + 25           #  TypeError
"Edad: " + str(25)      #
f"Edad: {25}"           #

# Índice de tipo incorrecto
my_list["0"]            #  TypeError
```

```
my_list[0]              #

# Operaciones entre tipos incompatibles
"3" + 5                 #  TypeError
int("3") + 5            #
```

Errores de Valor

```
# Conversión inválida
int("hello")            #  ValueError
int("42")               #

# Índice fuera de rango
my_list = [1, 2, 3]
my_list[10]             #  IndexError

# Key inexistente
my_dict = {"a": 1}
my_dict["b"]            #  KeyError
my_dict.get("b")        #  Retorna None
my_dict.get("b", 0)     #  Retorna 0
```

Errores Lógicos Comunes

```
# Comparar con = en lugar de ==
if x = 5:               #  SyntaxError
if x == 5:              #

# Modificar lista mientras se itera
for item in my_list:
    my_list.remove(item)  #  Comportamiento inesperado

for item in my_list.copy():  #
    my_list.remove(item)

# Mutable como parámetro por defecto
def add_item(item, items=[]):  #  Bug
    items.append(item)
    return items

def add_item(item, items=None):  #
    if items is None:
        items = []
    items.append(item)
    return items

# División entera inesperada
7 / 2   # 3.5 (división normal)
7 // 2  # 3 (división entera)
```

Comparación de Flotantes

```
# Comparación directa
0.1 + 0.2 == 0.3         #  False!
```

```python
# Usar isclose
import math
math.isclose(0.1 + 0.2, 0.3)  #  True

# O comparar con tolerancia
abs((0.1 + 0.2) - 0.3) < 1e-9  #  True
```

Recursos Relacionados

- **Siguiente**: Data Structures Cheat Sheet
- **Documentación oficial**: docs.python.org
- **PEP 8 - Style Guide**: pep8.org

Cheat Sheet creado para el Bootcamp Python Zero to Hero - 2026